D Y PATIL UNIVERSITY
NAVI MUMBAI

Subject Name :

Unit No: 3

Unit Name : Exception Handling using Python

Faculty Name :Prachi Abhijeet Surve

D Y PATIL
UNIVERSITY
NAVI MUMBAI

Unit No.:3

Faculty Name :Prachi Abhijeet Surve

Index

Lecture No & Topic Name	Slide No
Exception Handling using Python	
Introduction to Syntax,	
Errors,	
Exceptions,	
Handling Exceptions,	
User defined exceptions	
Clean up actions (Try … Catch…Finally),	
Predefined Clean up actions	

D Y PATIL UNIVERSITY
NAVI MUMBAI

Lecture No: 1

Exception Handling using Python

An exception is an unexpected event that occurs during program execution. For example,

```
divide_by_zero = 7 / 0
```

The above code causes an exception as it is not possible to divide a number by **0**.

Let's learn about Python Exceptions in detail.

Python Logical Errors (Exceptions)

Errors that occur at runtime (after passing the syntax test) are called exceptions or logical errors.

For instance, they occur when we

- try to open a file(for reading) that does not exist (`FileNotFoundError`)
- try to divide a number by zero (`ZeroDivisionError`)
- try to import a module that does not exist (`ImportError`) and so on.

Whenever these types of runtime errors occur, Python creates an exception object.

If not handled properly, it prints a traceback to that error along with some details about why that error occurred.

UNIVERSITY
NAVI MUMBAI

Let's look at how Python treats these errors:

```
divide_numbers = 7 / 0
print(divide_numbers)
```

Output

```
Traceback (most recent call last):
 File "<string>", line 1, in <module>
ZeroDivisionError: division by zero
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

Difference Between Errors and Exceptions

Errors and exceptions are both issues in a program, but they differ in severity and handling. Let's see how:

- **Error:** Serious problems in the program logic that cannot be handled. Examples include syntax errors or memory errors.
- **Exception:** Less severe problems that occur at runtime and can be managed using exception handling (e.g., invalid input, missing files).

D Y PATIL UNIVERSITY
NAVI MUMBAI

Example: This example shows the difference between a syntax error and a runtime exception.

```
# Syntax Error (Error)
print("Hello world"  # Missing closing parenthesis

# ZeroDivisionError (Exception)
n = 10
res = n / 0
```

Explanation: A syntax error stops the code from running at all, while an exception like ZeroDivisionError occurs during execution and can be caught with exception handling.

D Y PATIL UNIVERSITY
NAVI MUMBAI

Introduction to Syntax and Errors

In Python, there are two main types of errors:

- **Syntax Errors:** Also known as parsing errors. These occur when the Python interpreter cannot understand your code because it violates the rules of the language (e.g., a missing colon or unmatched parentheses).
- **Logical Errors:** The code runs, but it produces the wrong result.

Exceptions

Even if a statement is syntactically correct, it may cause an error when executed. These "runtime errors" are called **Exceptions**.

- **Examples:** `ZeroDivisionError` (dividing by zero), `TypeError` (adding a string to an integer), and `FileNotFoundError`.

D Y PATIL UNIVERSITY
NAVI MUMBAI

Handling Exceptions (The try...except block)

To prevent the program from terminating, we use the `try` and `except` blocks.

- **`try` block:** Contains the code that *might* throw an exception.
- **`except` block:** Contains the code that runs *if* an exception occurs.

Syntax and Usage

Python provides four main keywords for handling exceptions: try, except, else and finally each plays a unique role. Let's see syntax:

```
try:
    # Code
except SomeException:
    # Code
else:
   # Code
finally:
  # Code
```

- **try:** Runs the risky code that might cause an error.
- **except:** Catches and handles the error if one occurs.
- **else:** Executes only if no exception occurs in try.
- **finally:** Runs regardless of what happens useful for cleanup tasks like closing files.

D Y PATIL UNIVERSITY
NAVI MUMBAI

Example: This code attempts division and handles errors gracefully using try-except-else-finally.

```python
try:
    n = 0
    res = 100 / n

except ZeroDivisionError:
    print("You can't divide by zero!")

except ValueError:
    print("Enter a valid number!")

else:
    print("Result is", res)

finally:
    print("Execution complete.")
```

Output

```
You can't divide by zero!
Execution complete.
```

Explanation: try block attempts division, except blocks catch specific errors, else block executes only if no errors occur, while finally block always runs, signaling end of execution.

D Y PATIL UNIVERSITY
NAVI MUMBAI

Python Catching Exceptions:

When working with exceptions in Python, we can handle errors more efficiently by specifying the types of exceptions we expect. This can make code both safer and easier to debug.

1. Catching Specific Exceptions

2. Catching Multiple Exceptions

3. Catch-All Handlers and Their Risks

D Y PATIL
UNIVERSITY
NAVI MUMBAI

1. Catching Specific Exceptions

Catching specific exceptions makes code to respond to different exception types differently. It precisely makes your code safer and easier to debug. It avoids masking bugs by only reacting to the exact problems you expect.

Example: This code handles ValueError and ZeroDivisionError with different messages.

```
try:
    x = int("str")  # This will cause ValueError
    inv = 1 / x   # Inverse calculation

except ValueError:
    print("Not Valid!")

except ZeroDivisionError:
    print("Zero has no inverse!")
```

Output

```
Not Valid!
```

Explanation: A ValueError occurs because "str" cannot be converted to an integer. If conversion had succeeded but x were 0, a ZeroDivisionError would have been caught instead.

D Y PATIL UNIVERSITY
NAVI MUMBAI

2. Catching Multiple Exceptions

We can catch multiple exceptions in a single block if we need to handle them in the same way or we can separate them if different types of exceptions require different handling.

Example: This code attempts to convert list elements and handles ValueError, TypeError and IndexError.

```
a = ["10", "twenty", 30]  # Mixed list of integers and strings
try:
    total = int(a[0]) + int(a[1])  # 'twenty' cannot be converted to int

except (ValueError, TypeError) as e:
    print("Error", e)

except IndexError:
    print("Index out of range.")
```

Output

```
Error invalid literal for int() with base 10: 'twenty'
```

Explanation: The ValueError is raised when trying to convert "twenty" to an integer. A TypeError could occur if incompatible types were used, while IndexError would trigger if the list index was out of range.

D Y PATIL UNIVERSITY
NAVI MUMBAI

3. Catch-All Handlers and Their Risks

Sometimes we may use a catch-all handler to catch any exception, but it can hide useful debugging info.

Example: This code tries dividing a string by a number, which causes a TypeError.

```
try:
    res = "100" / 20 # Risky operation: dividing string by number

except ArithmeticError:
    print("Arithmetic problem.")

except:
    print("Something went wrong!")
```

Output

```
Something went wrong!
```

Explanation: A TypeError occurs because you can't divide a string by a number. The bare except catches it, but this can make debugging harder since the actual error type is hidden. Use bare except only as a last-resort safety net.

D Y PATIL UNIVERSITY
NAVI MUMBAI

Built-in Exceptions

- Commonly occurring exceptions are usually defined in the compiler/interpreter.
- **These are called built-in exceptions.**
- Python's standard library is an extensive collection of built-in exceptions that deals with the commonly occurring errors (exceptions) by providing the standardized solutions for such errors.
- On the occurrence of any built-in exception, the appropriate exception handler code is executed which displays the reason along with the raised exception name.
- The programmer then has to take appropriate action to handle it.
- Some of the commonly occurring built-in exceptions that can be raised in Python are explained in Table 1.1.

D Y PATIL
UNIVERSITY
NAVI MUMBAI

Table 1.1 Built-in exceptions in Python

S. No	Name of the Built-in Exception	Explanation
1.	SyntaxError	It is raised when there is an error in the syntax of the Python code.
2.	ValueError	It is raised when a built-in method or operation receives an argument that has the right data type but mismatched or inappropriate values.
3.	IOError	It is raised when the file specified in a program statement cannot be opened.
4	KeyboardInterrupt	It is raised when the user accidentally hits the Delete or Esc key while executing a program due to which the normal flow of the program is interrupted.
5	ImportError	It is raised when the requested module definition is not found.
6	EOFError	It is raised when the end of file condition is reached without reading any data by input().
7	ZeroDivisionError	It is raised when the denominator in a division operation is zero.
8	IndexError	It is raised when the index or subscript in a sequence is out of range.
9	NameError	It is raised when a local or global variable name is not defined.
10	IndentationError	It is raised due to incorrect indentation in the program code.
11	TypeError	It is raised when an operator is supplied with a value of incorrect data type.
12	OverFlowError	It is raised when the result of a calculation exceeds the maximum limit for numeric data type.

D Y PATIL UNIVERSITY
NAVI MUMBAI

Raising Exceptions

- Each time an error is detected in a program, the Python interpreter raises (throws) an exception.
- Exception handlers are designed to execute when a specific exception is raised.
- Programmers can also forcefully raise exceptions in a program using the raise and assert statements.
- Once an exception is raised, no further statement in the current block of code is executed.
- So, raising an exception involves interrupting the normal flow execution of program and jumping to that part of the program (exception handler code) which is written to handle such exceptional situations.
- The raise statement can be used to throw an exception.
- The syntax of raise statement is: **raise exception-name[(optional argument)]**
- The argument is generally a string that is displayed when the exception is raised.

D Y PATIL UNIVERSITY
NAVI MUMBAI

Basic Syntax:

raise ExceptionType("Error message")

Example: This code raises a ValueError if an invalid age is given.

```
def set(age):
    if age < 0:
        raise ValueError("Age cannot be negative.")
    print(f"Age set to {age}")

try:
    set(-5)
except ValueError as e:
    print(e)
```

Output

```
Age cannot be negative.
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

User-defined Exceptions in Python with Examples

Steps to Create and Use User-Defined Exceptions

Follow these steps to create and use User-Defined Exceptions in Python:

- **Define a New Exception Class:** Create a new class that inherits from Exception or any of its subclasses.
- **Raise the Exception:** Use the raise statement to raise the user-defined exception when a specific condition occurs.
- **Handle the Exception:** Use try-except blocks to handle the user-defined exception.

```
class ValueTooSmallError(Exception):
    pass

# Usage
raise ValueTooSmallError("The value is below the limit!")
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

Step 1: Define the Custom Exception Class

Start by creating a new class inherited from the built-in exception class or its subclass. The new class will be the custom exception and can have additional logic and properties as required.

Example:

```
class MyCustomError(Exception):
    pass
```

Key Points to Know

- **Defining a Class:** We can define a class using the standard Python class syntax. For a custom exception, we define an empty class body using the pass statement.
- **Inheritance:** Custom exceptions inherit from built-in exceptions, sharing the same behavior and attributes.

D Y PATIL UNIVERSITY
NAVI MUMBAI

Step 2: Initialize the Exception

We use the __init__ method to provide custom error messages or to initialize attributes. This helps us add specific details about the error when we raise an exception.

Example:

```
class InvalidAgeError(Exception):
    def __init__(self, age, message="Age can be between 21 and 40"):
        self.age = age
        self.message = message
        super().__init__(self.message)
```

Key Points to Know

- **Default Message:** We can override the default message when raising an exception.
- **Attributes:** We can define different attributes, such as age and message, to provide specific information about the error.
- **Initialization:** We use the __init__ method to initialize the attributes and super()__init__(self.message) to ensure that the base exception class is initialized properly with the error message.

D Y PATIL UNIVERSITY
NAVI MUMBAI

Step 3: Override Methods of the Parent Class

We override the __str__() method to customize how the error message appears when the exception is printed or logged.

Example:

```
class InvalidAgeError(Exception):
    def __init__(self, age, message="Age can be between 21 and 40"):
        self.age = age
        self.message = message
        super().__init__(self.message)

    def __str__(self):
        return f"{self.message}. Provided age: {self.age}"
```

Key Points to Know

- **Custom Message:** The customized message includes more relevant and specific information about the exception.
- **__str__Method:** It returns the string representation of the exception, which is displayed when the exception is printed.

D Y PATIL UNIVERSITY
NAVI MUMBAI

Benefits of User-Defined Exceptions in Python

User-defined exceptions offer several benefits that help us in Python programming.

Here are the top advantages of user-defined exceptions:

1. Error Handling

We can handle errors more efficiently with user-defined exceptions. When we define custom exceptions, we can handle each type of error separately and ensure effective debugging.

2. Code Reusability

We can reuse user-defined exceptions across multiple modules and projects, which allows us to reuse the code.

3. Code Readability

With user-defined exceptions, we have a clear idea of error conditions that may occur in the code. Using descriptive names for exceptions helps us better understand the context and purpose of each exception.

D Y PATIL UNIVERSITY
NAVI MUMBAI

Example: In this example, we create a custom exception InvalidAgeError to ensure that age values fall within a valid range (0–120).

```python
# Step 1: Define a custom exception class
class InvalidAgeError(Exception):
    def __init__(self, age, msg="Age must be between 0 and 120"):
        self.age = age
        self.msg = msg
        super().__init__(self.msg)

    def __str__(self):
        return f'{self.age} -> {self.msg}'

# Step 2: Use the custom exception in your code
def set_age(age):
    if age < 0 or age > 120:
        raise InvalidAgeError(age)
    else:
        print(f"Age set to: {age}")

# Step 3: Handling the custom exception
try:
    set_age(150)  # This will raise the custom exception
except InvalidAgeError as e:
    print(e)
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

Real-World Example: Invalid Email Error

```python
class InvalidEmailError(Exception):
    def __init__(self, email, msg="Invalid email format"):
        self.email = email
        self.msg = msg
        super().__init__(self.msg)

    def __str__(self):
        return f"{self.email} -> {self.msg}"

def set_email(email):
    if "@" not in email:
        raise InvalidEmailError(email)
    print(f"Email set to: {email}")

try:
    set_email("userexample.com")
except InvalidEmailError as e:
    print(e)
```

Explanation:

- A new exception class InvalidEmailError is defined to validate email addresses.
- If the given email does not contain "@", the exception is raised.
- The try-except block catches the error and prints the formatted message.

Output

```
userexample.com -> Invalid email format
```

User-defined exceptions should be considered in the following scenarios:

- Representing specific errors in an application (e.g., InvalidAgeError, DatabaseConnectionError).
- Providing clearer and more descriptive error messages.
- Handling a group of related errors separately using except.

D Y PATIL UNIVERSITY
NAVI MUMBAI

Output

```
150 -> Age must be between 0 and 120
```

Explanation:

- InvalidAgeError class inherits from Exception. It defines an __init__ method to accept age and message.
- The __str__ method returns a readable string representation of the error.
- In set_age(), if the age is outside the valid range (0–120), the exception is raised.
- The try-except block catches the exception and prints the error message.

D Y PATIL UNIVERSITY
NAVI MUMBAI

Need for Exception Handling

Following are some of the important points regarding exceptions and their handling:

- Python categorises exceptions into distinct types so that specific exception handlers (code to handle that particular exception) can be created for each type.
- Exception handlers separate the main logic of the program from the error detection and correction code.
- The segment of code where there is any possibility of error or exception, is placed inside one block.
- The code to be executed in case the exception has occurred, is placed inside another block.
- These statements for detection and reporting the exception do not affect the main logic of the program.
- The compiler or interpreter keeps track of the exact position where the error has occurred.
- Exception handling can be done for both user-defined and built-in exceptions.

D Y PATIL UNIVERSITY
NAVI MUMBAI

Advantages and Disadvantages of exception handling:

Advantages

Below are some benefits of using exception handling:

- **Improved reliability:** Programs don't crash on unexpected input.
- **Separation of concerns:** Error-handling code stays separate from business logic.
- **Cleaner code:** Fewer conditional checks scattered in code.
- **Helpful debugging:** Tracebacks show exactly where the problem occurred.

Disadvantages

Exception handling have some cons as well which are listed below:

- **Performance overhead:** Handling exceptions is slower than simple condition checks.
- **Added complexity:** Multiple exception types may complicate code.
- **Security risks:** Poorly handled exceptions might leak sensitive details

Clean-up Actions (try...except...finally)

Python provides a way to ensure certain code runs regardless of whether an error occurred or not.

- `else` **block:** Runs only if **no** exceptions were raised in the `try` block.
- `finally` **block:** Always executes, making it ideal for "clean-up" tasks like closing a file or a database connection.

We use of **try** statement which has an optional clause - **"finally"** to perform clean up actions, that must be executed under all conditions.

Cleanup actions:

- Before leaving the **try** statement, **"finally"** clause is always executed, whether an exception is raised or not.
- These are clauses which are intended to define clean-up actions that must be executed under all circumstances.
- Whenever an exception occurs and is not being handled by the **except** clause, first **finally** will occur and then the error is raised as default

D Y PATIL UNIVERSITY
NAVI MUMBAI

Code works normally and clean-up action is taken at the end

```python
# Python code to illustrate
# clean up actions
def divide(x, y):
    try:
        # Floor Division : Gives only Fractional Part as Answer
        result = x // y
    except ZeroDivisionError:
        print("Sorry ! You are dividing by zero ")
    else:
        print("Yeah ! Your answer is:", result)
    finally:
        print("I'm finally clause, always raised !! ")

# Look at parameters and note the working of Program
divide(3, 2)
```

Output :

```
Yeah ! Your answer is : 1
I'm finally clause, always raised !!
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

Code raise error and is carefully handled in the except clause. Note that Clean-up action is taken at the end.

```python
# Python code to illustrate
# clean up actions
def divide(x, y):
    try:
        # Floor Division : Gives only Fractional Part as Answer
        result = x // y
    except ZeroDivisionError:
        print("Sorry ! You are dividing by zero ")
    else:
        print("Yeah ! Your answer is:", result)
    finally:
        print("I'm finally clause, always raised !! ")

# Look at parameters and note the working of Program
divide(3, 0)
```

Output :

```
Sorry ! You are dividing by zero
I'm finally clause, always raised !!
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

Code, raise error but we don't have any except clause to handle it. So, clean-up action is taken first and then the error(by default) is raised by the compiler.

```python
# Python code to illustrate
# clean up actions
def divide(x, y):
    try:
        # Floor Division : Gives only Fractional Part as Answer
        result = x // y
    except ZeroDivisionError:
        print("Sorry ! You are dividing by zero ")
    else:
        print("Yeah ! Your answer is:", result)
    finally:
        print("I'm finally clause, always raised !! ")

# Look at parameters and note the working of Program
divide(3, "3")
```

Output :

```
I'm finally clause, always raised !!
```

Error:

```
Traceback (most recent call last):
  File "C:/Users/DELL/Desktop/Code.py", line 15, in
    divide(3, "3")
  File "C:/Users/DELL/Desktop/Code.py", line 7, in divide
    result = x // y
TypeError: unsupported operand type(s) for //: 'int' and 'str'
```

Pre-defined Clean Up Actions:

- In Python, pre-defined clean up actions are mostly used to close the files.
- When we are executing some operations like read and write in the file, generally files are left open for long period of time.
- This may not cause any serious problem in simple scripts, but its sure causes trouble while working with larger applications.
- This pre-defined clean up actions automatically closes the file after its suite finishes, even though an exception raised on the way. It can used by the keyword “with”.

Python Code:

```
with open('C:\\Users\\sriharshithaghali\\Documents\\mynotes\\examples.txt', 'r') as
f:
    read_data = f.read()
    print(read_data)
f.closed
```

Output:

True

D Y PATIL UNIVERSITY
NAVI MUMBAI

Discuss the concept of clean-up actions in Python exception handling and explain their importance.

- In Python, **clean-up actions** refer to the operations performed to release resources or restore the program to a safe state after executing a block of code, whether an exception occurs or not.
- These actions ensure that resources such as **files, network connections, or memory** are properly closed or released.
- Clean-up actions are usually implemented using the **`finally` block** in exception handling.

Clean-up Actions Using `finally`

The **`finally` block** is always executed after the `try` and `except` blocks, regardless of whether an exception occurs.

Syntax

```
try:
  # Code that may raise an exception
except ExceptionType:
  # Code to handle exception
finally:
  # Clean-up actions
```

The `finally` block ensures that important tasks such as **closing files or releasing system resources** are executed.

Example Program

```
try:
  file = open("data.txt", "r")
  content = file.read()
  print(content)
except FileNotFoundError:
  print("File not found.")
finally:
  file.close()
  print("File closed successfully.")
```

Explanation

1. The `try` block attempts to open and read the file.
2. If the file does not exist, the `except` block handles the **FileNotFoundError**.
3. The `finally` block executes **file.close()**, ensuring the file is closed even if an error occurs.

D.Y. PATIL UNIVERSITY NAVI MUMBAI

Predefined Clean-up Actions

Python also provides **predefined clean-up mechanisms**, such as the **`with` statement**, which automatically manages resources.

Example

```
with open("data.txt", "r") as file:
  content = file.read()
  print(content)
```

Here, the file is **automatically closed** after the block execution.

Importance of Clean-up Actions

Clean-up actions are important for the following reasons:

1. **Resource Management** – Ensures files, database connections, and network resources are properly closed.
2. **Prevents Memory Leaks** – Releases memory and system resources after use.
3. **Program Stability** – Maintains the correct state of the program even if errors occur.
4. **Improves Reliability** – Ensures that critical tasks are always executed.
5. **Better Error Handling** – Prevents resource conflicts in long-running programs.

- Clean-up actions play a vital role in **Python exception handling** by ensuring that resources are properly released and the program remains stable.
- The **`finally` block** and constructs like the **`with` statement** help guarantee that necessary clean-up operations are executed regardless of exceptions.

D Y PATIL UNIVERSITY NAVI MUMBAI

Analyze how exception handling helps in developing robust and reliable Python applications.

- Exception handling in Python is a mechanism used to **detect, manage, and respond to runtime errors** that occur during program execution.
- Instead of terminating the program abruptly, exception handling allows the program to **handle errors gracefully and continue execution**.
- This makes Python applications more **robust, reliable, and user-friendly**.

Concept of Exception Handling

In Python, exceptions are handled using `try`, `except`, `else`, **and** `finally` **blocks**.

Basic Syntax

```
try:
  # code that may cause an exception
except ExceptionType:
  # code to handle the exception
else:
  # executes if no exception occurs
finally:
  # cleanup actions
```

This structure allows programmers to separate **normal program logic from error-handling logic**.

D Y PATIL UNIVERSITY
NAVI MUMBAI

How Exception Handling Improves Robustness

1. Prevents Program Crashes

Without exception handling, runtime errors cause the program to stop abruptly.
Exception handling allows the program to **catch errors and continue execution**.

Example:

```
try:
  num = int(input("Enter a number: "))
  result = 10 / num
  print(result)
except ZeroDivisionError:
  print("Division by zero is not allowed.")
```

2. Improves Error Detection and Debugging

Exception handling provides **clear error messages**, making it easier to identify and fix issues during program development.

Example:

```
except ValueError:
  print("Invalid input. Please enter a numeric value.")
```

3. Ensures Proper Resource Management

Using the `finally` **block**, resources such as files or network connections can be properly closed even when an error occurs.

Example:

```
try:
  file = open("data.txt","r")
except FileNotFoundError:
  print("File not found")
finally:
  print("Operation completed")
```

4. Enhances User Experience

Exception handling prevents technical error messages from being shown to users and instead displays **friendly and understandable messages**.

Example:

Invalid input. Please enter a valid number.

5. Supports Custom Error Handling

Python allows the creation of **user-defined exceptions**, enabling developers to handle application-specific errors effectively.

Example:

```
class AgeError(Exception):
  pass
```

6. Maintains Program Stability

Exception handling ensures that unexpected situations are handled properly, allowing the program to **recover from errors without affecting the overall system**.

D Y PATIL UNIVERSITY NAVI MUMBAI

Advantages of Exception Handling

- Improves **program reliability**

- Prevents **abrupt termination**
- Helps in **maintaining data integrity**
- Makes programs **easier to debug and maintain**
- Ensures **efficient resource utilization**

- Exception handling is an essential feature of Python that enables developers to **manage runtime errors efficiently**.
- By preventing program crashes, ensuring proper resource management, and providing meaningful error messages, exception handling helps in building **robust, reliable, and maintainable Python applications**.

D Y PATIL UNIVERSITY
NAVI MUMBAI

Summary

- **Syntax errors or parsing errors** are detected when we have not followed the rules of the particular programming language while writing a program.
- When syntax error is encountered, Python displays the name of the error and a small description about the error.
- The execution of the program will start only after the syntax error is rectified.
- **An exception is a Python object that represents an error.**
- **Syntax errors are also handled as exceptions.**
- The exception needs to be handled by the programmer so that the program does not terminate abruptly.
- When an exception occurs during execution of a program and there is a built-in exception defined for that, the error message written in that exception is displayed.
- The programmer then has to take appropriate action and handle it.
- Some of the commonly occurring built-in exceptions are
  - SyntaxError,
  - ValueError,
  - IOError,
  - KeyboardInterrupt,
  - ImportError,
  - EOFError,
  - ZeroDivisionError,
  - IndexError,
  - NameError,
  - IndentationError,
  - TypeError,and
  - OverFlowerror.
- When an error is encountered in a program, Python interpreter raises or throws an exception.

D Y PATIL UNIVERSITY
NAVI MUMBAI

Summary

- **Exception Handlers are the codes that are designed to execute when a specific exception is raised.**
- Raising an exception involves interrupting the normal flow of the program execution and jumping to the exception handler.
- Raise and assert statements are used to raise exceptions.
- The process of exception handling involves writing additional code to give proper messages or instructions to the user.
- This prevents the program from crashing abruptly.
- The additional code is known as an exception handler.
- An exception is said to be caught when a code that is designed to handle a particular exception is executed.
- An exception is caught in the try block and handles in except block.
- The statements inside the finally block are always executed regardless of whether an exception occurred in the try block or not.

D Y PATIL UNIVERSITY
NAVI MUMBAI

Thank You

D Y PATIL
UNIVERSITY
NAVI MUMBAI